

Модель программы OpenMP

Технология OpenMP основана на многопоточности современных ОС и наиболее эффективна на симметричных мультипроцессорных системах с общей памятью.

Процесс ОС

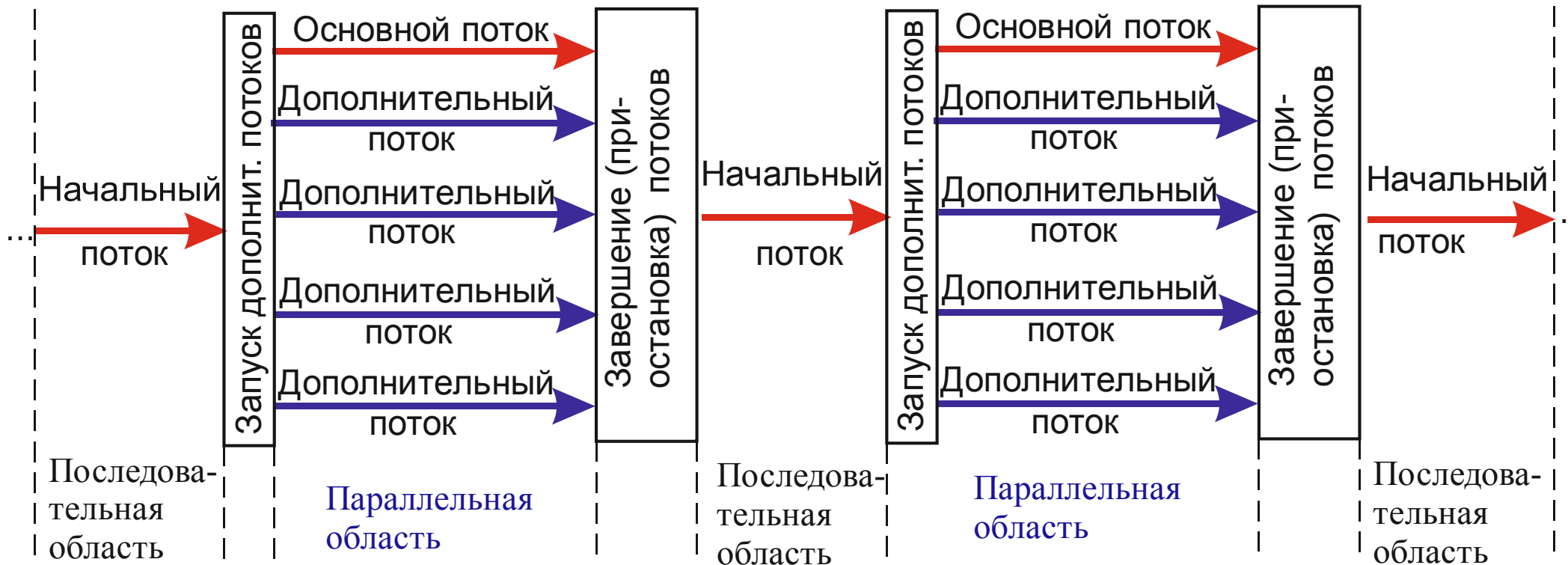


Рис. 1

Активизация режима компиляции с поддержкой средств OpenMP

MS Visual C++

```
cl -FeProga -MD -O2 -openmp Sources.cpp
```

GNU C++

```
g++ -o Proga -O3 -fopenmp Sources.cpp
```

Intel C++ (Windows)

```
icl -FeProga -MD -O3 -Qopenmp Sources.cpp
```

Intel C++ (Linux)

```
icc -o Proga -O3 -openmp Sources.cpp
```

Средства OpenMP

- **Директивы**

`#pragma omp <имя-директивы> [<параметры>]`
`<программный блок>`

1) для создания параллельных областей; 2) для распределения вычислительной работы; 3) для синхронизации потоков

- **Функции**

`#include <omp.h>`

Пример: `void omp_set_num_threads(int num);`

- **Переменные окружения**

Пример – Windows/cmd: использовать 4 потока OpenMP

`set OMP_NUM_THREADS=4`

Пример – UNIX/bash: использовать 4 потока OpenMP

`export OMP_NUM_THREADS=4`

Задание параметров при помощи функций имеет приоритет над переменными окружения.

Создание параллельной области

```
#pragma omp parallel [параметр[[,] параметр]...]
< программный блок >
```

При входе в параллельную область порождаются новые `OMP_NUM_THREADS-1` потоков, каждый поток получает свой уникальный номер, причём порождающий поток получает номер 0 и становится основным потоком группы («мастером»).

Остальные потоки получают в качестве номера целые числа с 1 до `OMP_NUM_THREADS-1`. Все потоки исполняют один и тот же код, соответствующий параллельной области. Количество потоков, выполняющих данную параллельную область, остаётся неизменным до момента выхода из области. При выходе из параллельной области производится неявная синхронизация и уничтожаются (вернее – приостанавливаются) все потоки, кроме породившего.

Возможные параметры (или опции):

- `if(условие)`

– выполнение параллельной области по условию. Вхождение в параллельную область осуществляется только при выполнении некоторого условия. Если условие не выполнено, то директива не срабатывает и продолжается обработка программы в прежнем режиме;

- `num_threads(целочисленное выражение)`

– явное задание количества потоков, которые будут выполнять параллельную область; по умолчанию выбирается последнее значение, установленное с помощью функции `omp_set_num_threads()`, или значение переменной `OMP_NUM_THREADS`;

- `default(private|firstprivate|shared|none)`
— всем переменным в параллельной области, которым явно не назначен класс (хранения), будет назначен класс `private`, `firstprivate` или `shared` соответственно; `none` означает, что всем переменным в параллельной области класс должен быть назначен явно; в языке C задаются только варианты `shared` или `none`;
- `private(список)` — задаёт список переменных, для которых порождается локальная копия в каждом потоке; начальное значение локальных копий переменных из списка не определено;
- `firstprivate(список)` — задаёт список переменных, для которых порождается локальная копия в каждом потоке; локальные копии переменных инициализируются значениями этих переменных в основном потоке;
- `shared(список)` — задаёт список переменных, общих для всех потоков;

- `copyin(список)` – задаёт список переменных, объявленных как `threadprivate`, которые при входе в параллельную область инициализируются значениями соответствующих переменных в основном потоке;

- `reduction(оператор:список)` – задаёт оператор и список общих переменных; для каждой переменной создаются локальные копии в каждом потоке; локальные копии инициализируются соответственно типу оператора (для аддитивных операций – 0 или его аналоги, для мультипликативных операций – 1 или её аналоги); над локальными копиями переменных после выполнения всех операторов параллельной области выполняется заданный оператор; оператор это:

`+, *, -, &, |, ^, &&, ||`

порядок выполнения операторов не определён, поэтому результат может отличаться от запуска к запуску

Сокращенная запись директив

Если внутри параллельной области содержится:

- только один параллельный цикл;
- только одна конструкция `sections`
- только одна конструкция `workshare`

то можно использовать укороченную запись:

- `parallel for`
- `parallel sections`
- `parallel workshare`

При этом допустимо указание всех опций этих директив, за исключением опции `nowait`.

Вспомогательные функции и переменные среды

- Возврат в вызвавшем потоке астрономического времени в секундах (отсчитываемого с некоторого момента)
`double omp_get_wtime(void);`
- Возврат в вызвавшем потоке разрешения таймера в секундах
`double omp_get_wtick(void);`
- Задание числа потоков, исполняющих параллельную область
`set OMP_NUM_THREADS=n (Windows/cmd)`
`export OMP_NUM_THREADS=n (UNIX/bash)`
`void omp_set_num_threads(int num);`
- Разрешить динамически изменять количество потоков, используемых для выполнения параллельной области
`export OMP_DYNAMIC=true (Windows/cmd)`
`set OMP_DYNAMIC=true (UNIX/bash)`
`void omp_set_dynamic(int num);`

- Возврат значения переменной OMP_DYNAMIC
`int omp_get_dynamic(void);`
- Возврат максимально допустимого числа потоков для использования в следующей параллельной области
`int omp_get_max_threads(void);`
- Возврат количества процессоров, доступных для использования программе пользователя на момент вызова.
`int omp_get_num_procs(void);`
- Параллельные области могут быть вложенными; по умолчанию вложенная параллельная область выполняется одним потоком. Если вложенный параллелизм разрешён (и поддерживается реализацией OpenMP), то каждый поток, в котором встретится описание параллельной области, породит для её выполнения новую группу потоков. Сам породивший поток станет в новой группе основным потоком.

- Для разрешения вложенного параллелизма

```
export OMP_NESTED=true (UNIX/bash)
set OMP_NESTED=true (Windows/cmd)
void omp_set_nested(int nested)
```
- Возврат значения переменной OMP_NESTED

```
int omp_get_nested(void);
```
- Определение номера потока в текущей параллельной секции

```
int omp_get_thread_num(void);
```
- Определение нахождения в последовательной либо параллельной области

```
int omp_in_parallel(void);
```

Директива `single`

– Если в параллельной области какой-либо участок кода должен быть выполнен лишь один раз

```
#pragma omp single [параметр [[,] параметр]...]
```

Какой именно поток будет выполнять выделенный участок программы, не специфицируется. Один поток будет выполнять данный фрагмент, а все остальные потоки будут ожидать завершения её работы, если только не указан параметр (опция) `nowait`.

Возможные параметры (опции):

- `private(список)`

– задаёт список переменных, для которых порождается локальная копия в каждом потоке; начальное значение локальных копий переменных из списка не определено;

- `firstprivate(список)`

– задаёт список переменных, для которых порождается локальная копия в каждом потоке; локальные копии переменных инициализируются значениями этих переменных в основном потоке;

- **copyprivate**(список)

— после выполнения потока, содержащего конструкцию `single`, новые значения переменных списка будут доступны всем одноименным частным переменным (`private` и `firstprivate`), описанным в начале параллельной области и используемым всеми её потоками; опция не может использоваться совместно с опцией `nowait`; переменные списка не должны быть перечислены в опциях `private` и `firstprivate` данной директивы `single`;

- **nowait**

— после выполнения выделенного участка происходит неявная барьерная синхронизация параллельно работающих потоков: их дальнейшее выполнение происходит только тогда, когда все они достигнут данной точки; если в подобной задержке нет необходимости, опция `nowait` позволяет потокам, уже дошедшим до конца участка, продолжить выполнение без синхронизации с остальными.

Директива **master**

Позволяет выделить участок кода, который будет выполнен только основным. Остальные потоки просто пропускают данный участок и продолжают работу с оператора, расположенного следом за ним. Неявной синхронизации данная директива не предполагает.

```
#pragma omp master
```

Модель данных

Переменные в параллельных областях программы разделяются на два основных класса:

- **shared**

Общие переменные; все потоки видят одну и ту же переменную.

Общая переменная всегда существует лишь в одном экземпляре для всей области действия и доступна всем нитям под одним и тем же именем.

- **private**

Локальные, или приватные переменные; каждая нить видит свой экземпляр данной переменной. Объявление локальной переменной вызывает порождение своего экземпляра данной переменной (того же типа и размера) для каждого потока. По умолчанию, все переменные, порождённые вне параллельной области, при входе в эту область остаются общими (**shared**). Исключение – счетчики итераций в цикле. Переменные, порождённые внутри параллельной области, по умолчанию являются локальными (**private**).

Дополнительные возможности:

- `firstprivate(список)` – задаёт список переменных, для которых порождается локальная копия в каждом потоке; локальные копии переменных инициализируются значениями этих переменных в основном потоке;

- Директива

`#pragma omp threadprivate(список)`

указывает, что переменные из списка должны быть размножены с тем, чтобы каждая нить имела свою локальную копию.

Позволяет сделать локальные копии для статических переменных, которые по умолчанию являются общими. Если на локальные копии ссылаются в разных параллельных областях, то для сохранения их значений необходимо, чтобы не было объемлющих параллельных областей, количество нитей в обеих областях совпадало, а переменная `OMP_DYNAMIC` была установлена в `false` с начала первой области до начала второй.

- Если необходимо переменную, объявленную как `threadprivate`, инициализировать значением размножаемой переменной из основного потока, то на входе в параллельную область можно использовать опцию `copyin`. Если значение локальной переменной или переменной, объявленной как `threadprivate`, необходимо переслать от одной нити всем, работающим в данной параллельной области, для этого можно использовать опцию `copyprivate` директивы `single`.

- Параметр (опция)

`lastprivate(список)`

позволяет присвоить переменным, перечисленным в списке, значение с последней итерации цикла так, как если бы он выполнялся последовательно.

Распределение вычислительной работы

Низкоуровневое распределение

- Все потоки в параллельной области нумеруются последовательными целыми числами от 0 до $N-1$, где N — количество потоков, выполняющих данную область.
- Можно распределять вычислительную работу между потоками, используя следующие функции:

`int omp_get_thread_num(void);`

— позволяет потоку получить свой уникальный номер в текущей параллельной области;

`int omp_get_num_threads(void);`

— позволяет потоку получить количество потоков в текущей параллельной области.

Параллельные циклы

#pragma omp for [<параметр> [[,]] <параметр> ...]

Возможные параметры

private(<список переменных>)

firstprivate(<список переменных>)

lastprivate(<список переменных>)

reduction(<оператор>:<список>)

schedule(type [, <размер>])

где type может быть static

dynamic

guided

auto

runtime

collapse(<число вложенных циклов>)

ordered

nowait

Возможные ограничения

Программист отвечает за то, что итерации цикла не имеют информационных зависимостей.

```
for([<целочисленный тип>] j=<инвариант цикла>;  
    j{< , > , <=, >=}<инвариант цикла>;  
    j{+,-}=<инвариант цикла>)
```

Балансировка вычислительной нагрузки

set OMP_SCHEDULE="dynamic,1" - Windows/Cmd

export OMP_SCHEDULE="dynamic,1" - UNIX/bash

Прототипы соответствующих функций из <omp.h>:

```
void omp_set_schedule(omp_sched_t type, int chunk);
```

```
void omp_get_schedule(omp_sched_t *type, int *chunk);
```

Параллелизм на уровне задач

```
#pragma omp sections [<параметр>...]
{
#pragma omp section
<блок программы>
#pragma omp section
<блок программы>
.....
}
```

Возможные значения параметров

```
private(<список переменных>)
firstprivate(<список переменных>)
lastprivate(<список переменных>)
reduction(<оператор>:<список>)
nowait
```

Выделение отдельных подзадач

```
#pragma omp task [<параметр> [[,] <параметр>...]]
```

```
<блок программы>
```

Возможные параметры:

```
if(<условие>)
```

```
untied
```

```
default(private | firstprivate | shared | none)
```

```
private(<список переменных>)
```

```
firstprivate(<список переменных>)
```

```
shared(<список переменных>)
```

Для гарантии завершения подзадач в точке вызова

```
#pragma omp taskwait
```

Средства синхронизации потоков

Барьерная синхронизация

`#pragma omp barrier`

Блок в цикле, выполняющийся, как в послед. версии

`#pragma omp ordered`

`<блок программы>`

Блок операторов относится к самому внутреннему циклу.

Критические секции

`#pragma omp critical [(<имя критической секции>)]`

`<блок программы>`

В фиксированный момент времени в критической секции может находиться лишь один поток, а остальные — ожидают его выхода. Неименованные критические секции ассоциируются с одним и тем же именем. Задание имени уменьшает число блокировок.

Блокировка обновления общих переменных

```
#pragma omp atomic
```

```
<переменная>=<выражение>;
```

```
#pragma omp atomic
```

```
<переменная>++;
```

```
#pragma omp atomic
```

```
<переменная>--;
```

```
#pragma omp atomic
```

```
++<переменная>;
```

```
#pragma omp atomic
```

```
--<переменная>;
```

и т.п.

Замки (задвижки)

- Могут находиться в одном из трех состояний:
 неинициализирован
 разблокирован
 заблокирован
- Существует два типа замков: простые и множественные

Инициализация:

```
void omp_init_lock(omp_lock_t *lock);
```

```
void omp_init_nest_lock(omp_nest_lock_t *lock);
```

Перевод в неинициализированное состояние

```
void omp_destroy_lock(omp_lock_t *lock);
```

```
void omp_destroy_nest_lock(omp_nest_lock_t *lock);
```

Захват замка

```
void omp_set_lock(omp_lock_t *lock);
```

```
void omp_set_nest_lock(omp_nest_lock_t *lock);
```

В последнем случае коэф-т захваченности увеличивается на 1.

Освобождение замка

```
void omp_unset_lock(omp_lock_t *lock);
```

```
void omp_unset_nest_lock(omp_nest_lock_t *lock);
```

В последнем случае коэффициент захваченности уменьшается на 1. Освобождение множественного замка происходит, если его коэффициент захваченности обнуляется.

Неблокирующая попытка захвата замка

```
int omp_test_lock(omp_lock_t *lock);
```

```
int omp_test_nest_lock(omp_nest_lock_t *lock);
```

Для простого замка возвращает 1, а для множественного – новый коэф-т захваченности. Если захват невозможен, возвращает 0.

Синхронизация образа оперативной памяти

`#pragma omp flush [<список переменных>]`

- Неявно присутствует в директиве `barrier` и в операторе присваивания, соотв. директиве `atomic`
- На входе и выходе области действия директив `parallel`, `critical`, `ordered`
- На выходе областей распределения работ, если не задан параметр `nowait`
- Перед порождением и после завершения любой задачи (`task`)
- При вызовах (если замок захватывается или освобождается)

`void omp_set_lock(omp_lock_t *lock);`

`void omp_set_nest_lock(omp_nest_lock_t *lock);`

`void omp_unset_lock(omp_lock_t *lock);`

`void omp_unset_nest_lock(omp_nest_lock_t *lock);`

`int omp_test_lock(omp_lock_t *lock);`

`int omp_test_nest_lock(omp_nest_lock_t *lock);`